Tarefa3: Lab04Tarefa04 – Algoritmos baseados em Densidade

Nome: Leonardo Scholl Sternberg

Curso: Algoritmos para Agrupamento de Dados

Especialização em Inteligência Artificial para Engenheiros

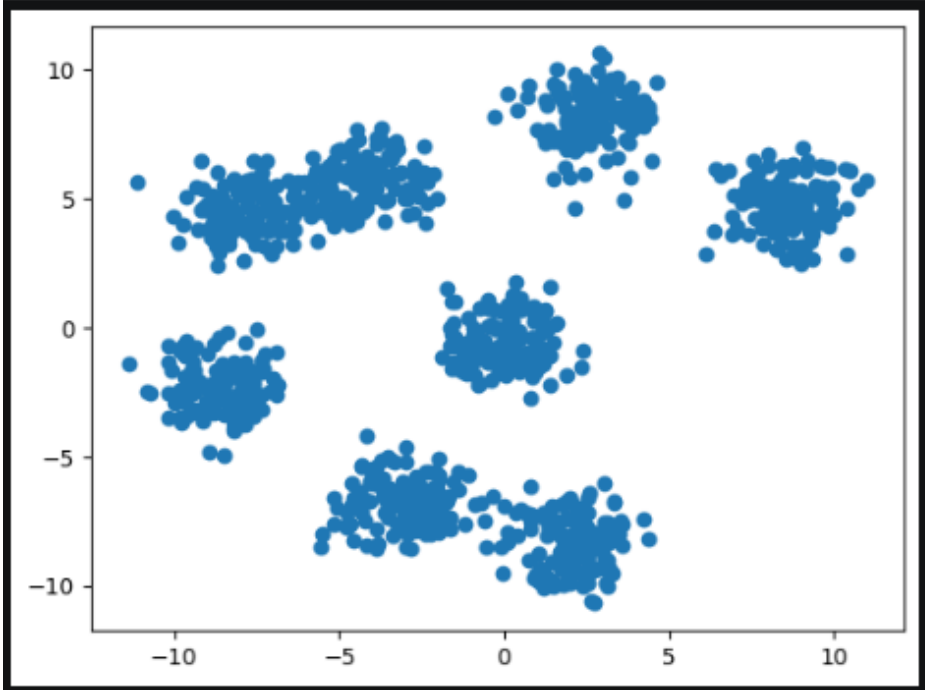
Avaliação do Código

1: Configuração, Carga e Preparação dos Dados

Este bloco inicial estabelece o ambiente de trabalho e gera o conjunto de dados sintético para os testes de densidade:

```
# Carrega as bibliotecas e funções necessárias
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
%matplotlib inline
```

```
# Gera uma base de dados aleatória

X, y = make_blobs(n_samples=1000, centers=8, n_features=2, random_state=800)
plt.scatter(X[:,0], X[:,1])
plt.show()
```



- **Bibliotecas:** Importação do DBSCAN do Scikit-Learn para o agrupamento, “make_blobs” para a geração de dados e matplotlib para visualização.
- **Geração de Dados:** Utilizou-se a função “make_blobs” para criar 1000 amostras bidimensionais distribuídas em 8 centros distintos (centers=8), com um estado aleatório fixado em 800 para reprodutibilidade.
- **Visualização Inicial:** O gráfico de dispersão inicial mostra nuvens de pontos com diferentes densidades e proximidades, o que representa um desafio ideal para algoritmos baseados em densidade como o DBSCAN.

2: Implementação e Teste do Hiperparâmetro Núm_mín

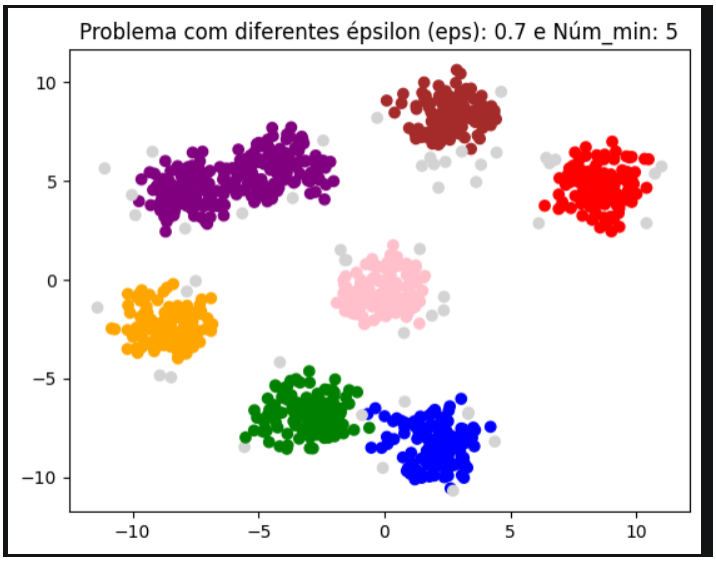
Nesta etapa, o algoritmo DBSCAN é executado iterativamente para avaliar como a exigência de vizinhos mínimos altera a percepção de clusters e a identificação de ruídos:

```
# Testa o algoritmo DBSCAN com ε = 0.7, e  os  seguintes  valores para Núm_mín: 5, 10, 19, 20, 25

# Define uma lista de cores fixas para ruído (-1) e clusters (0, 1, 2, 3, 4, 5, 6, 7, 8)
# ruído = lightgray
# cluster 0 = red
# cluster 1 = blue
# cluster 2 = green
# cluster 3 = orange
# cluster 4 = purple
# cluster 5 = brown
# cluster 6 = pink
# cluster 7 = cyan
# cluster 8 = yellow
fixed_colors = ['lightgray', 'red', 'blue', 'green', 'orange', 'purple', 'brown', 'pink', 'cyan', 'yellow']

ep = 0.7
Num_min_values = [5, 10, 19, 20, 25]

for num_min in Num_min_values:
    db = DBSCAN(eps=ep, min_samples=num_min, metric='euclidean')
    labels = db.fit_predict(X)
    plot_colors = [fixed_colors[label + 1] if label != -1 else fixed_colors[0] for label in labels]
    plt.scatter(X[:,0], X[:,1], c=plot_colors)
    plt.title('Problema com diferentes épsilon (eps): ' + str(ep) + ' e Núm_min: ' + str(num_min))
    plt.show()
```

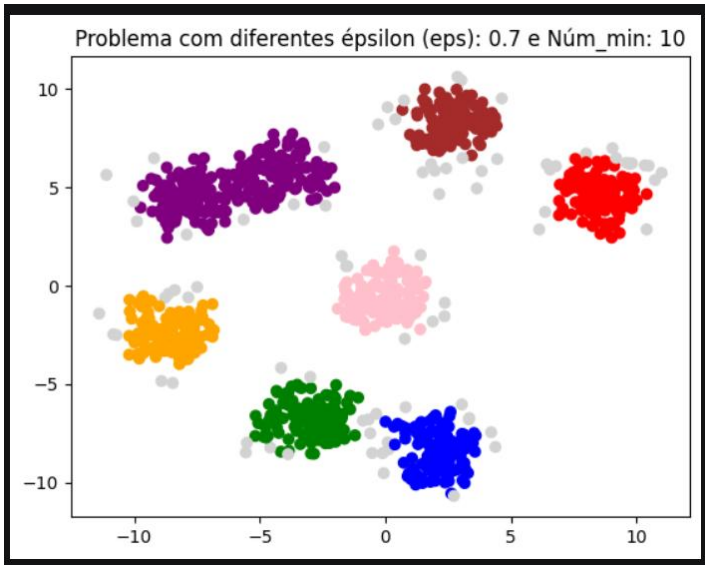
- **Parâmetros Fixos:** O raio de vizinhança foi mantido em ε = 0,7 e a métrica de distância utilizada foi a 'euclidean'.
- **Variação de Núm_mín:** O código percorre a lista de valores [5, 10, 19, 20, 25].
- **Mapeamento de Cores:** Foi definida uma lista de cores fixas onde o valor -1 (ruído) é sempre representado em cinza claro (lightgray), permitindo distinguir claramente o que o algoritmo não conseguiu agrupar.

3: Análise Visual dos Resultados e Fenômeno de Fragmentação

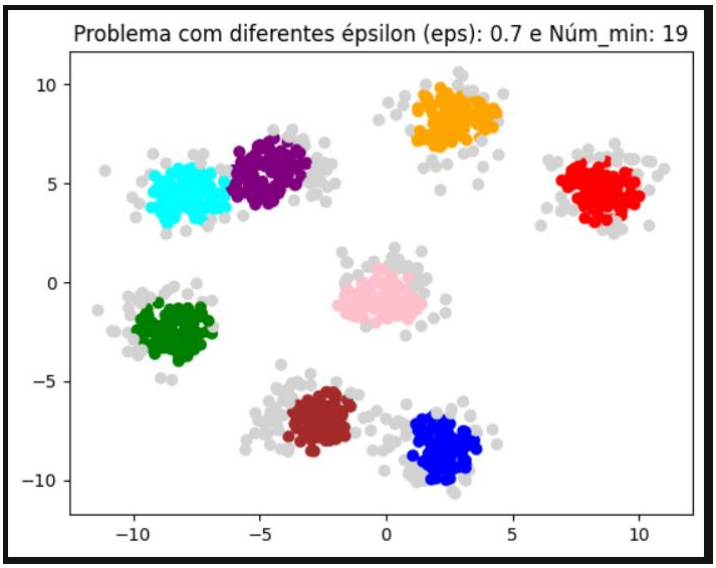
Abaixo, descreve-se o comportamento do algoritmo conforme o aumento do hiperparâmetro min_samples:



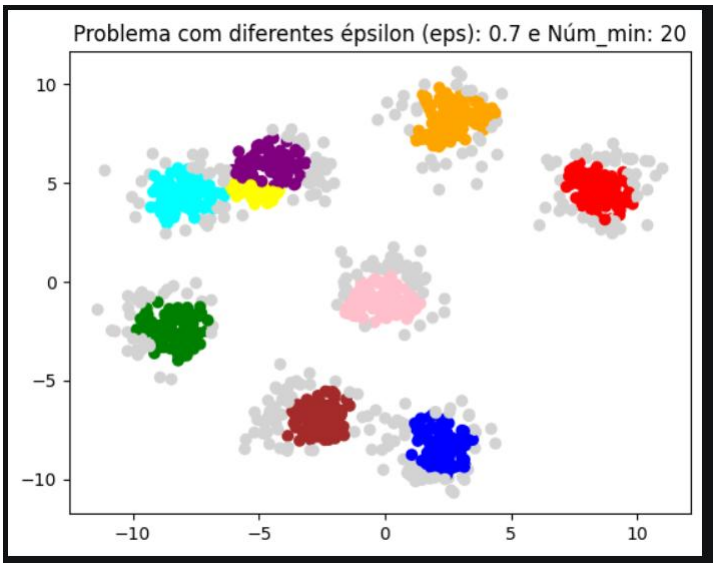
Núm_mín = 5: O algoritmo identifica **7 clusters**. Note que, nesta configuração, dois dos "blobs" originais localizados na parte superior esquerda (em roxo) estão fundidos em um único cluster grande. O ε de 0,7 é grande o suficiente para conectá-los, e a exigência de apenas 5 vizinhos não é suficiente para separá-los.



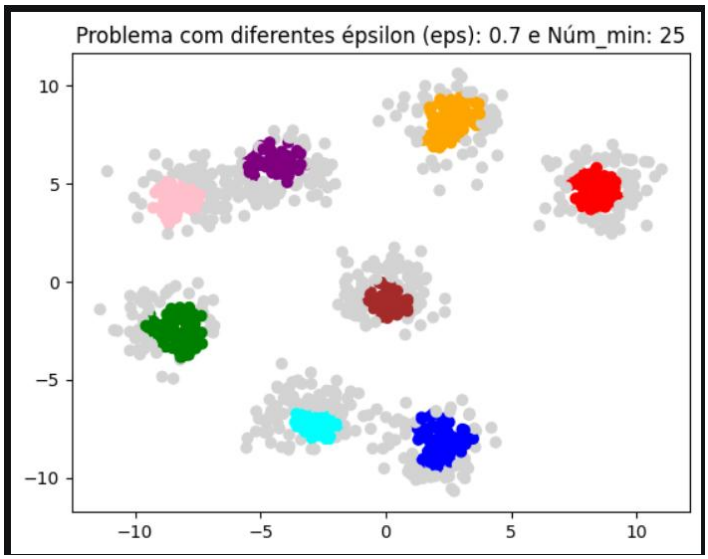
Núm_mín = 10: O cenário permanece com 7 clusters. Embora a quantidade de ruído (pontos cinzas) aumente levemente nas bordas, a densidade entre os dois grupos fundidos no topo ainda é alta o suficiente para que o DBSCAN os trate como uma única estrutura contínua.



Núm_mín = 19: Aqui ocorre uma mudança drástica, resultando em 8 clusters. Com a exigência de 19 vizinhos, a "ponte" de baixa densidade que unia os dois grupos roxos superiores se quebra. Um deles permanece roxo e o outro torna-se ciano. Pela primeira vez, o algoritmo consegue separar os 8 grupos originais, embora o custo seja um aumento significativo de pontos descartados como ruído (cinza).



Núm_mín = 20: O algoritmo identifica 9 clusters. Como discutimos, a exigência de 20 vizinhos causou a fragmentação de um dos agrupamentos centrais. O que era um cluster único agora está dividido entre roxo e amarelo, pois os pontos de ligação internos não atingiram a densidade mínima exigida.



Núm_mín = 25: O número de clusters cai para 6 identificados. Com uma exigência de 25 vizinhos, a sensibilidade é tão alta que dois clusters inteiros desaparecem (tornam-se totalmente cinzas/ruído) e outros continuam fragmentados ou reduzidos apenas aos seus núcleos mais compactos.

Discussão e Considerações Finais

A análise demonstra que o valor ideal para este conjunto de dados, buscando a separação original, é **Núm_mín = 19**.

Fusão vs. Fragmentação: Valores baixos (5 e 10) falharam em separar grupos próximos, fundindo-os (7 clusters). Valores altos (20) fragmentaram grupos coesos (9 clusters).

O ponto de equilíbrio: O valor de **19** testado capaz de "quebrar" as pontes de densidade indesejadas e revelar os **8 clusters** gerados originalmente, apesar de gerar mais ruído nas periferias.

Conclusão Técnica: O experimento prova que o DBSCAN exige uma sintonia fina. Diferente do K-Means (Tarefa anterior) ou do Agrupamento Hierárquico, onde você define ou corta o número de grupos, aqui o número de clusters é uma **consequência direta** da densidade local definida por ε e min_samples.